

High Performance Computing – Introductory Task 1

Orazgali Azamat, BDA-2402

Part 1: Basic HPC Concepts

1. What is High Performance Computing?

High Performance Computing (HPC) is the use of aggregated, powerful computing resources (many processors/cores working together, sometimes across multiple machines) to solve computational problems that are too large, too complex, or too slow to run on an computer.

2. Give two examples of problems for which HPC may be useful.

- Weather and climate simulation, where scientists have to numerically solve physical models over huge 3D
- Working with self-hosted LLMs or VLMs instead of cloud LLMs. For example, using GPU clusters to handle large models like Qwen, Gemma, Kimi require tremendous GPU clusters

3. Explain the difference between:

- **Processor/core** – a single processing unit capable of executing instructions independently. A modern CPU chip usually has several cores
- **Node** – a single computer with its own CPU, memory, and possibly accelerators that acts as one unit within a larger system
- **Cluster** – a collection of many nodes connected by a network, working together and coordinated as a single system to solve large problems

4. Why are multiple processors useful for solving large computational problems?

A large problem can be split into smaller independent (or loosely dependent) sub-problems that are solved simultaneously on different processors, reducing total wall-clock time. This also allows problems too large for one machine's memory to be spread across the combined memory of many nodes. It is a basic principle of multithreading, where we can distribute a complex task into different threads to handle high loads.

5. Suppose a computer has 4 processors. Does this automatically mean that every program will run four times faster? Explain briefly.

No. Speedup depends on how much of the program can actually be parallelized. Any purely sequential part (setup, I/O, synchronization, combining results) cannot be sped up by adding processors.

Part 2: A Simple Computational Problem

I declared **array_sum.cpp** for the C++ implementation. It:

1. Creates a `std::vector<long long>` of size $N = 1,000,000$.
2. Initializes $a[i] = i$.
3. Increases every element by 1 ($a[i] = a[i] + 1$).
4. Sums all elements.
5. Prints the final sum.

Compiled and run result for $N = 1,000,000$:

```
Sum = 500000500000
```

Part 3: Thinking in Parallel

Dividing $N = 1,000,000$ elements among 4 processors:

Processor	First index	Last index
P0	0	249,999
P1	250,000	499,999
P2	500,000	749,999
P3	750,000	999,999

1. Can the four processors increase their parts of the array simultaneously?

Yes. Each processor only reads and writes its own disjoint slice of the array, so the four increment operations are fully independent and can run at the same time

2. Does P0 need the result produced by P1 before it can perform its calculations?

No. Since the update of each element only depends on its own previous value, P0's work on indices 0–249,999 does not depend on anything P1 computes on indices 250,000–499,999

3. Which part of this computation can therefore be performed in parallel?

The element-wise increment step can be fully parallelized, since it is a parallel operation with no dependencies between elements

4. What additional operation is needed to calculate one final sum of the entire array?

Each processor computes a partial sum of its own slice, and then these four partial sums must be combined (reduced)

Part 4: Small Experiment

Program: array_sum.cpp, compiled with `g++ -O2 -std=c++11` (MinGW-w64 16.1.0) and run on this machine. Only the increment loop (`a[i] = a[i] + 1`) was timed, as instructed.

Array size N	Execution time (seconds)
10^5	~ 0.000023
10^6	~ 0.00035
10^7	~ 0.0072

Raw repeated measurements (3 runs per N):

Compiler: `g++ (WinLibs MinGW-w64 16.1.0), flags: -O2 -std=c++11`

N = 100000

Time: 2.17e-05 seconds

Time: 2.39e-05 seconds

Time: 2.27e-05 seconds

N = 1000000

Time: 0.0003879 seconds

Time: 0.0003083 seconds

Time: 0.0003668 seconds

N = 10000000

Time: 0.0073208 seconds

Time: 0.007328 seconds

Time: 0.0071132 seconds

Part 5: Short Discussion

1. What happens to the execution time when N increases?

The execution time increases roughly in proportion to N, since the loop performs one increment operation per element – i.e., the work (and therefore the time) grows linearly ($O(N)$) with the array size. Going from 10^5 to 10^6 to 10^7 increases the time by roughly a factor of 10 each step, consistent with the measurements above

2. Why might using several processors be useful when N becomes very large?

Splitting the array across several processors lets the independent increment operations run concurrently, reducing the wall-clock time roughly by the number of

processors used and lets the data be spread across the combined memory of multiple nodes.

3. What possible additional work may be required when several processors cooperate on the same problem?

We can include dividing the data among processors, communicating/distributing the sub-arrays, synchronizing processors so they don't interfere with shared data, and combining their partial into one final answer